Real-Time Chat Application Documentation

1. Introduction

Project Title

Real-Time WebSocket Chat Application

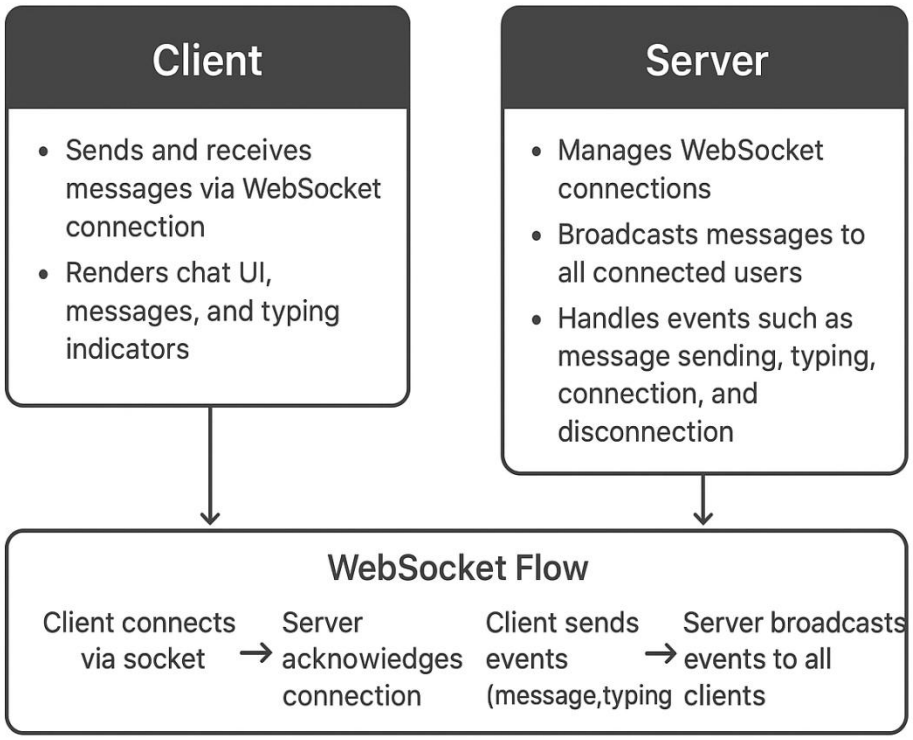
Objective

To build a real-time chat system that allows multiple users to send and receive messages instantly using WebSockets.

Technology Stack Used

- **Frontend:** React.js, HTML, CSS
- **Backend:** Node.js, Express.js
- **Realtime Engine:** Socket.IO (WebSocket-based)

2. Architecture Diagram



3. Backend Implementation

Code Snippets

- Socket connection handling
- Broadcasting messages
- Typing indicator event

Explanation of Events Handled

- `connection`
- `disconnect`
- `sendMessage`
- `receiveMessage`
- `typing`

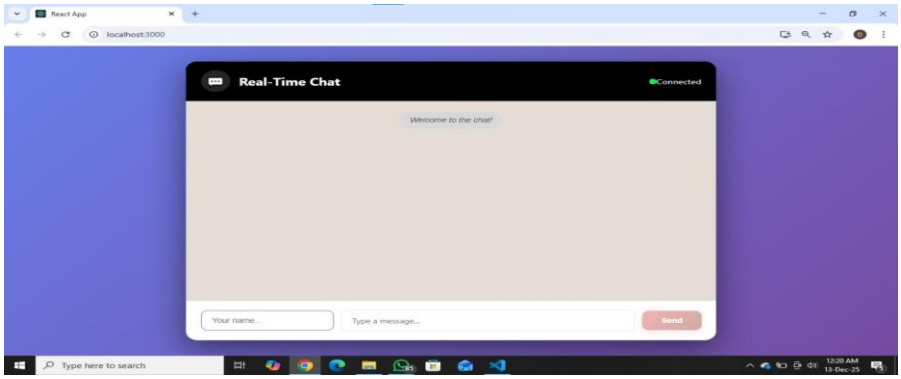
Message Broadcasting Logic

- Server receives a message → emits it to all connected clients.

4. Frontend Implementation

UI Flow

Screenshots of Chat Interface



Connection & Message Handling

- Connect to server using Socket.IO client.
- Manage message state using React.

5. Feature Explanation

List of Features

- Real-time messaging
- Typing indicator
- Connection status
- Multi-user chat support

How Each Feature Works Internally

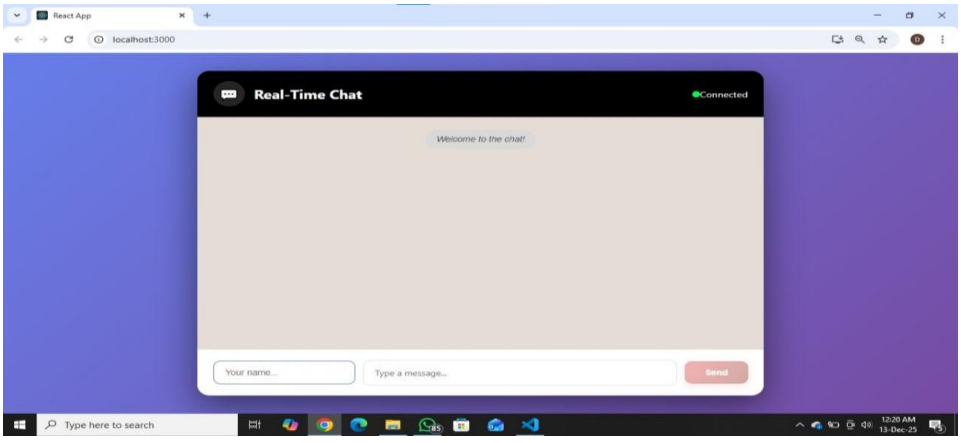
- Event listeners and emitters
- State updates in React
- Broadcast logic in Socket.IO

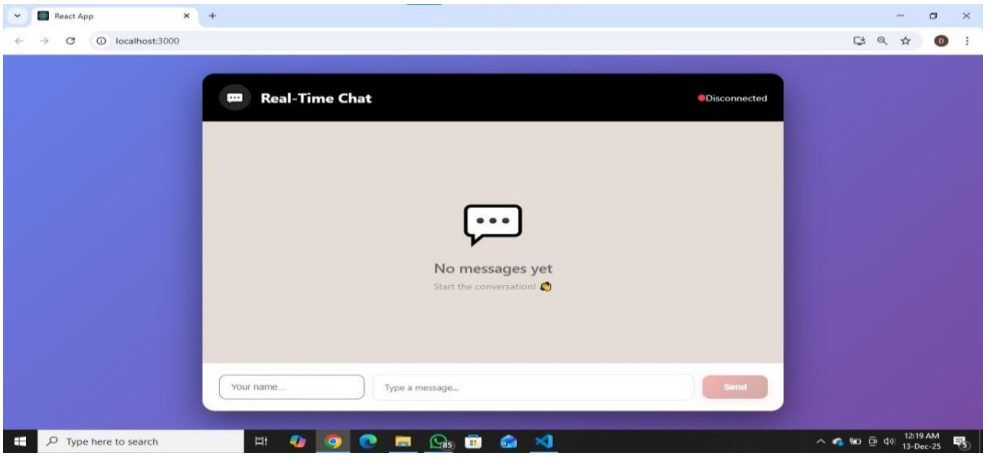
6. Testing Section

- Test sending messages
- Test multiple users in different browsers
- Test typing indicator
- Test connection/disconnection
- Test room join (if implemented)

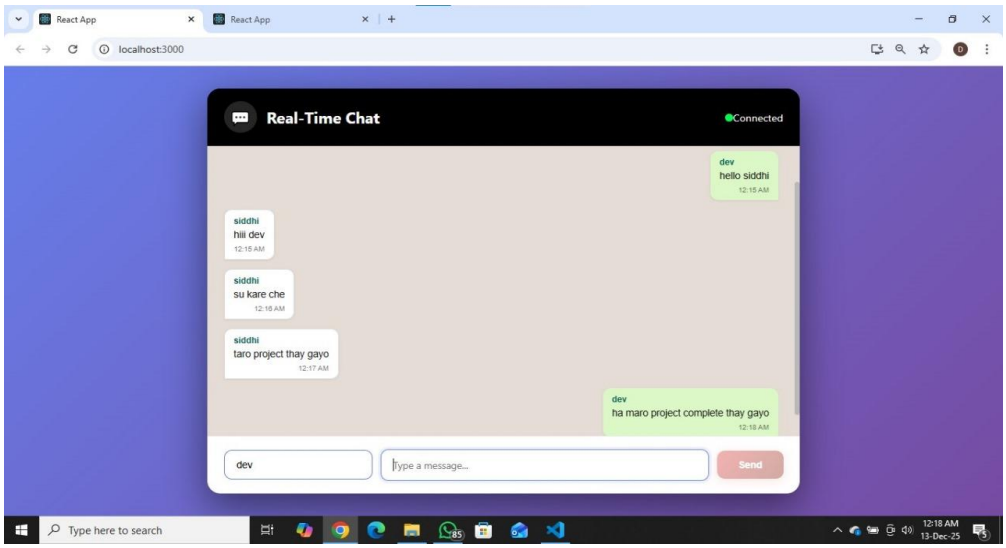
7. Output Screenshots

- Chat UI

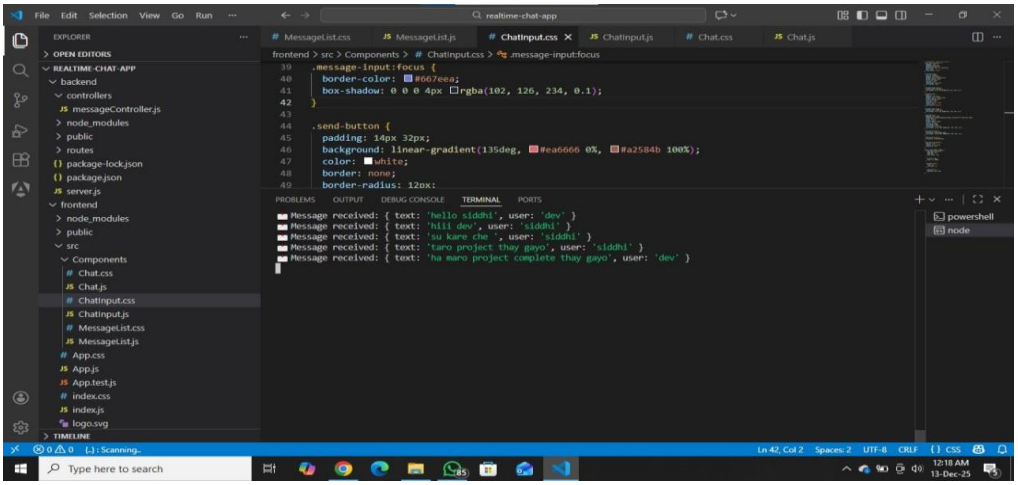




- Message logs



- Terminal/WebSocket logs



8. Conclusion

- Real-time communication using WebSockets
- Managing events in frontend and backend

Importance of WebSocket Technology

- Enables instant data exchange
- Used in chat apps, games, live dashboards

Future Scope

- Video chat integration
- Authentication system
- Push notifications
- Chat history storage

Thank You...!